Vision du Projet : WhytChat## Concept CentralWhytChat dépasse le simple chatbot pour devenir un écosystème d'Intelligences Artificielles interconnectées. L'objectif est de créer une dynamique de réflexion autonome entre les modèles.## Fonctionnalités Clés### 1. Communication Inter-IA (Local / Cross-Topic)- **Principe** : Les modèles d'IA ne sont pas isolés dans une seule conversation. Ils peuvent analyser et communiquer transversalement sur différents sujets actifs.- **Scénario d'exemple** : - _Discussion A_ : L'utilisateur parle du **Climat**. - _Discussion B_ : L'utilisateur parle de **Politique**. - **Interaction** : Les IAs s'interrogent mutuellement pour trouver des points communs, des causalités ou des paradoxes entre le Climat et la Politique.- **Objectif** : Faire émerger des réflexions "non-humaines", des angles d'analyse inédits et une profondeur de synthèse supérieure.### 2. Communication Inter-IA (Réseau LAN)- **Principe** : Capacité pour une instance de WhytChat de découvrir et de communiquer avec d'autres instances sur le réseau local.- **Objectif** : Permettre l'échange de connaissances ou de "pensées" entre différentes machines sans dépendre d'un serveur central cloud, favorisant un réseau de neurones distribué et privé.---### 3. Architecture "Mixture of Agents" (Multi-Vendeurs)- **Concept** : Utiliser la diversité des modèles (Microsoft, Google, Meta, Alibaba) pour enrichir la réflexion.- **Structure** : - **Synthétiseur (Main)** : Un modèle équilibré (ex: Qwen 2.5 3B) qui orchestre et rédige. - **Experts (SLM - Small Language Models)** : Des petits modèles spécialisés sollicités pour des tâches précises.   - _Expert Logique_ : Microsoft Phi-3.5 (Réputé pour le raisonnement pur).   - _Expert Créatif_ : Google Gemma 2 2B (Réputé pour la rédaction).   - _Expert Critique_ : Meta Llama 3.2 3B (Réputé pour la nuance et la sécurité).- **Avantage** : Chaque constructeur ayant ses propres biais et données d'entraînement, leur combinaison offre une "sagesse des foules" artificielle supérieure à un seul gros modèle.---_Ce document sert de référence pour l'évolution fonctionnelle et l'architecture de l'application._

Internationalization (i18n) GuideThis document defines the minimal, framework agnostic i18n conventions for WhytChat. It works with our current static HTML and scales to React later.## Goals- Keep it simple (no runtime dependency by default).- Deterministic keys and predictable fallbacks.- Easy to port to React/i18next if needed.## File Layout- Frontend translations live in `Frontend/translations/` as flat JSON files: - `en.json` – the canonical source - `fr.json`, `es.json`, ... – localized copiesExample `Frontend/translations/en.json`:```json{ "app.title": "WhytChat", "app.subtitle": "Minimal local chat", "btn.health": "Health", "btn.download": "Download model", "btn.start": "Start server", "btn.stop": "Stop server", "chat.placeholder": "Type message...", "chat.send": "Send", "status.downloading": "Downloading model...", "status.model_ready": "Model ready: {{path}}", "status.starting": "Starting server...", "status.started": "Server started on 127.0.0.1:8080", "status.stopped": "Server stopped", "error.generic": "Error: {{message}}"}```Notes:- Use dot separated keys (area.kind.action), all lowercase.- Use `{{...}}` placeholders for values that are interpolated at runtime.- English is the reference; other locales must keep identical keys.## Markup Conventions (Static HTML)- Add `data-i18n="key"` on elements to translate textContent.- For placeholder/value attributes, use `data-i18n-attr="placeholder|title|aria-label"` to list attributes to translate.Example snippet in `Frontend/index.html`:```html<h1 data-i18n="app.title"></h1><p data-i18n="app.subtitle"></p><button id="btn-health" data-i18n="btn.health"></button><input id="msg" data-i18n="chat.placeholder" data-i18n-attr="placeholder" /><button id="send" data-i18n="chat.send"></button>```## Runtime Loader (Vanilla JS)Add a tiny loader to:- Detect browser locale (`navigator.language`), normalize to `en`, `fr`, etc.- Fetch `Frontend/translations/<locale>.json`, fallback to `en.json`.- Apply translations to elements marked with `data-i18n` and `data-i18n-attr`.- Provide a simple `t(key, vars)` function.Example loader (drop into `<script>` after DOM is ready):```jsasync function loadLocale(locale) { const short = (locale || "en").split("-")[0]; async function fetchJson(lang) {   try {   return await (await fetch(`translations/${lang}.json`)).json();   } catch {   return null;   } } const en = await fetchJson("en"); const loc = (short !== "en" ? await fetchJson(short) : null) || {}; const dict = { ...en, ...loc }; function interpolate(str, vars) {   return str.replace(/{{\s*(\w+)\s*}}/g, (_, k) =>   k in (vars || {}) ? String(vars[k]) : ""   ); } window.t = (key, vars) => interpolate(dict[key] || key, vars); document.querySelectorAll("[data-i18n]").forEach((el) => {   const key = el.getAttribute("data-i18n");   el.textContent = window.t(key); }); document.querySelectorAll("[data-i18n-attr]").forEach((el) => {   const attrs = el.getAttribute("data-i18n-attr").split("|");   const key = el.getAttribute("data-i18n");   attrs.forEach((a) => el.setAttribute(a, window.t(key))); });}window.addEventListener("DOMContentLoaded", () => loadLocale(navigator.language));```## Using Translations in ScriptsPrefer `window.t(key, vars)` for dynamic messages:```jsout.textContent = t("status.model_ready", { path: p });out.textContent = t("error.generic", { message: String(e) });```## Adding New Text1. Pick a new key under the right namespace (e.g., `settings.language.save`).2. Add English string to `en.json`.3. Copy the same key to other locales with a temporary English value or a TODO.4. Update HTML/JS to reference the key.## Plurals (Minimal)- Keep simple: avoid grammar dependent plurals at first.- If needed, suffix keys: `inbox.item_one`, `inbox.item_many` and switch by count in JS.## Accessibility- Remember attributes: `aria-label`, `title`, `placeholder` using `data-i18n-attr`.- Ensure language changes don't remove context for screen readers.## Future: React/i18next- This structure maps naturally to i18next namespaces.- If migrating, reuse `translations/*.json` as resource bundles.## Testing Checklist- Missing keys: UI should show the key itself (easy to spot) or log a warning.- Fallback: disconnect `fr.json` to verify English fallback.- Encoding: files must be UTF 8 without BOM.

Project Structure — WhytChatCette arborescence sépare clairement UI (Frontend), logique/OS (Backend), et artefacts (modèles, binaires). Elle est contractuelle: toute génération/modification par l'agent doit s'y conformer.## Vue d'ensemble```WhytChat/ Frontend/ components/ pages/ styles/ translations/ public/ package.json tsconfig.json vite.config.ts Backend/ src-tauri/ Cargo.toml tauri.conf.json capabilities/ default.json icons/ src/ main.rs llama.rs llama_install.rs db.rs build.rs llama/ server/ # binaires llama.cpp/llama-server par OS/arch dll/ # bibliothèques natives requises models/ presets.json pack-sources.json packs/ # archives modèles téléchargées cache/ # extractions temporaires Shared/ # optionnel (types, schémas partagés) types/ schemas/ messages/ scripts/ dev.ps1 build.ps1 release.ps1 docs/ I18N_GUIDE.md PROJECT_STRUCTURE.md ARCHITECTURE.md SECURITY.md .github/ workflows/```## Rôles & Règles- Frontend: UI uniquement; appels `tauri.invoke` pour la logique.- Backend/src-tauri: commandes Tauri, processus llama, DB, I/O.- Backend/models: sources et index des modèles avec `sha256`; pas de code.- Backend/llama: binaires/runtime tiers isolés.- Capabilities: principe du moindre privilège (Tauri v2).## Événements & Progression (Tauri v2)- `download/progress` (global): `{ downloaded: u64, total?: u64, percent?: number }`.- `download/finished` (global): `{ path: string, sha256: string, verified: boolean, expected?: string }`.- `server_ready` (commande): renvoie `bool`; la UI active l'envoi si `true`.Notes:- Le hashing SHA-256 est calculé côté backend durant le téléchargement; la vérification n'est faite que si un `expected_sha256` est fourni à `download_model`.- La UI écoute les événements via `window.__TAURI__.event.listen`.Installation serveur LLaMA:- Utiliser la commande `install_server(backend?: "cpu" | "cuda-<ver>")` qui télécharge depuis les Releases officielles `ggml-org/llama.cpp` et extrait `llama-server.exe` dans `Backend/llama/server/`.- Événements émis: `server/install/progress`, `server/install/finished` (inclut `zip_sha256`). Pour une validation stricte, comparez la somme avec la valeur publiée sur la page de Release.## Build & Liaisons- `Backend/src-tauri/tauri.conf.json` → `build.distDir` pointe vers le build Frontend (ex: `../Frontend/dist`).- Sorties Tauri: `Backend/src-tauri/target/**` (non versionnées).- Icônes/ressources: `Backend/src-tauri/icons/`.## Contraintes de Conformité (Agent)- Respect strict de cette arborescence pour toute création/modification.- Toute déviation doit être explicitement validée puis répercutée ici.- Mettre à jour ce document avant/pendant toute refonte structurelle.

WhytChat — Plan d'Architecture Multi-Agents (MoA)## ObjectifConcevoir un système de conversation orchestrant un **LLM principal (Synthèse)** et plusieurs **agents experts (SLM)** hétérogènes (constructeurs différents) pour améliorer qualité, robustesse et diversité cognitive.## Architecture Fonctionnelle- **Utilisateur → Orchestrateur** (LLM principal) reçoit la requête.- **Phase Experts**: 2–4 agents spécialisés produisent des analyses ciblées (logique, créativité, critique/sécurité, factuel).- **Phase Synthèse**: LLM principal agrège les rapports et génère la réponse finale.## Modèles (candidats GGUF)- **Expert Logique**: Microsoft Phi-3.5 mini (≈3.8B, Q4_K_M), ou fallback Qwen2-1.5B.- **Expert Créatif**: Google Gemma 2 2B (licence à accepter) ou SmolLM3-3B (Apache-2.0).- **Expert Critique/Sécurité**: Llama 3.2 3B ou SmolLM3-3B.- **Synthétiseur (Main)**: Qwen2.5-3B ou SmolLM3-3B selon ressources.Références (extraits):- llama.cpp server flags: `--port`, `-c/--ctx-size`, `-ngl`, `--parallel`, `--metrics`, `--slots`.- Multi-instances: lancer plusieurs `llama-server` sur ports distincts; alternatives: RPC / proxy (llama-swap) ou mode séquentiel.## Contraintes & Stratégies- **Ressources**: multi-serveurs consomme RAM/VRAM; prévoir mode **séquentiel** (mêmes binaire & port, prompts différents) comme fallback.- **Licences**: Gemma 2 nécessite acceptation; fournir fallback libre (SmolLM3 / Qwen2).- **Perfs**: Activer `--metrics` pour observabilité; paralléliser via `reqwest` + `tokio` si multi-ports.## Design Backend (Tauri/Rust)- `AppState`: passer de `server: Option<Child>` à `servers: HashMap<u16, Child>`.- `start_server(model_path, port, ctx, ngl, parallel, metrics)` → lance un `llama-server` paramétré.- `stop_server(port)` → stoppe un serveur.- `server_ready(port)` → check santé.- `chat_single(port, messages)` → POST /v1/chat/completions.- `chat_multi(agents: [{port, system, user}], synth_port)`: - Exécute en parallèle `chat_single` pour chaque agent (messages injectés: system+user+historique filtré), - Concatène les rapports, - Appelle `chat_single` sur `synth_port` pour produire la réponse finale.## Design Frontend- Par conversation, **Config Agents**: liste d'experts {alias, port, modèle, prompts spécialisés} + **Synthétiseur**.- Bouton/flux "Débat" → 1 tour Experts → 1 tour Synthèse → réponse UI.- Historiser: stocker rapports agents pour audit.## Roadmap (V1 → V2)- **V1**: Orchestration séquentielle (mono-serveur), agents simulés par prompts spécialisés; robuste aux faibles ressources.- **V2**: Multi-serveurs parallèles (ports 8080/8081/8082), `tokio::join!`, `--metrics`.- **V3**: Découverte LAN (option), partage d'analyses entre pairs.## Tests & Observabilité- Smoke tests: start/stop servers, healthcheck.- Bench: latence/parallélisme, impact `-ngl` et `--ctx-size`.- Logs: Brain.log + métriques `/metrics`.